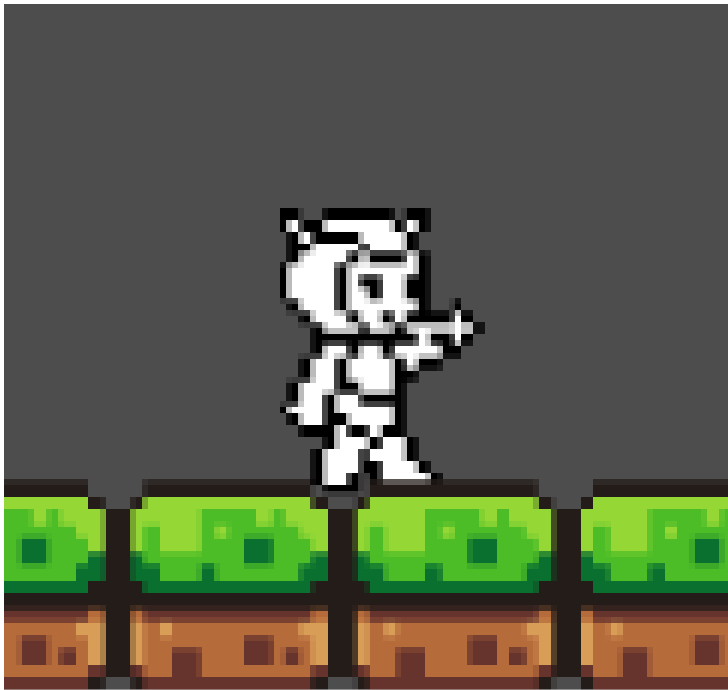


Projekt Spieleentwicklung: Omega Sprinter



Name: Nils Brück
Datum: 05.06.2026
Klasse: 11BGx3
Fach: Praktische Informatik
Fachlehrer: Danilo Magdeburg

Inhaltsverzeichnis

1	Einleitung	1
2	Planung	1
2.1	Produkt und Idee in eigenen Worten	1
2.2	Prototypen	1
2.3	Versionierung & Meilensteine	2
3	Entwürfe	2
3.1	Entwurf Horizontales Movement des Spielers	2
3.2	Entwurf: Input des Spielers	3
3.3	Entwurf: Gegnerinteraktion	3
4	Durchführung	4
4.1	Beschreiben des Ablaufs der Implementierung	4
4.2	Beschreibung des Spielablaufs, Screenshots und Codeausschnitte	4
4.3	Fehler und Behebung	11
4.4	Begründung zur Abweichung	11
5	Fazit und Ausblick	12
6	Quellen	12

Abbildungsverzeichnis

1	Prototyp Start/Ingame	1
2	Schuss Mechanik	1
3	Levelziel	2
4	Strukturgramm Horizontales Movement	2
5	Strukturgramm User Inputs	3
6	Strukturgramm Gegnerinteraktion	3
7	Levelauswahl Bildschirm	4
8	Level 1 Start	6
9	Level 1 Erster Gegner	6
10	Level 1 Gegner Abschießen	7
11	Level 1 Gegner Getroffen und Entfernt	8
12	Level 1 Liveball	8
13	Level 1 Levelende	9
14	Level 2 Checkpoint	9
15	Level 3 Stil	10
16	Level 4 Stil	10
17	Level 5	11

Listingsverzeichnis

1	Level-ende	5
2	Funktion Horizontales Movement	5
3	Funktionsabschnitt Sprung	5
4	Funktion Schuss des Players	7
5	Funktion Schuss des Geschosses	8
6	Respawn Funktion	9

1 Einleitung

Omega Sprinter ist ein 2D-Jump-and-Run Spiel mit Run-and-Gun Elementen. Das Spiel wurde im Rahmen des Schulunterrichts konzipiert und entwickelt. Vorlage für dieses Projekt sind die klassischen Mega Man Spiele, also Platforming und lineares Leveldesign, sowie die Möglichkeit zu schießen.

In der folgenden Dokumentation werden Spielidee, Entwurf, Durchführung und ein Fazit für das Projekt beschrieben.

2 Planung

2.1 Produkt und Idee in eigenen Worten

Grundidee des Spiels ist ein 2D-Jump-and-Run, vereint mit Run-and-Gun-Elementen. Das grundlegende Ziel ist es, das Levelende, ähnlich einer Flagge bei Super Mario, zu erreichen, um das Level abzuschließen. Außerdem sollen vereinzelt Sammelemente wie Münzen und Energiebälle zu finden sein. Der Schwerpunkt des Spiels wird es sein, lineare Level mit Platforming-Einlagen und verschiedenen Gegnerarten abzuschließen und nach Möglichkeit so viele Sammelobjekte wie möglich zu erhalten. Die HUD-Elemente sollen eher sparsam sein: ein Energietracker als Lebensanzeige in Form einer Zahl von 1 bis 100 und ein Münzzähler, der das gesamte Spiel hindurch hochzählt. In den Levels soll es Checkpoints geben, an denen ein Respawn nach dem Tod möglich ist. Damit das Sterben auch bestraft wird, verliert der Spieler pro Tod Teile seiner Münzen. Fallen diese auf Null, wird das Level beendet und der Spieler kommt zurück an den Levelanfang. Um das Spiel zu beenden, muss das letzte Level erreicht und darin eine finale, schwere Herausforderung bestanden werden. Grundlegend orientiert sich das Spiel an den originalen Mega Man Spielen.

2.2 Prototypen

Start und Idee in einem Level

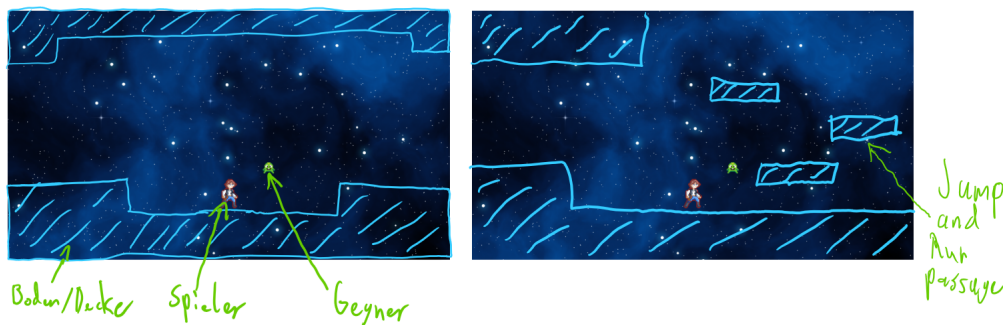


Figure 1: Prototyp Start/Ingame

Schuss Mechanik

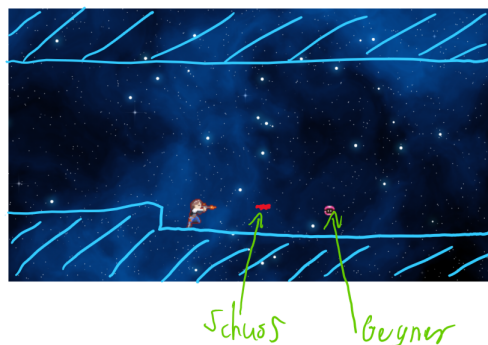


Figure 2: Schuss Mechanik

Ziel eines Levels

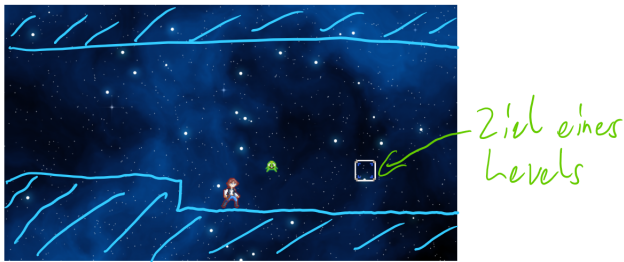


Figure 3: Levelziel

2.3 Versionierung & Meilensteine

Für die Versionierung wird Github und Git genutzt.

Meilensteine sind:

1. 07.05.26 Erstes simples Jump and Run Level bauen
2. 14.05.26 Einfügen von Gegnern und Levelabschluss
3. 21.05.26 Einfügen von Schuss Mechanik auf X-Achse
4. 28.05.26 Erweitern um mehr Level & Boss-Level
5. Je nach Zeit erweitern der Schuss Mechanik auf Y-Achse

3 Entwürfe

Diese Sektion behandelt die Strukturgramm Entwürfe für die grundlegendsten Mechaniken im Spiel.

3.1 Entwurf Horizontales Movement des Spielers

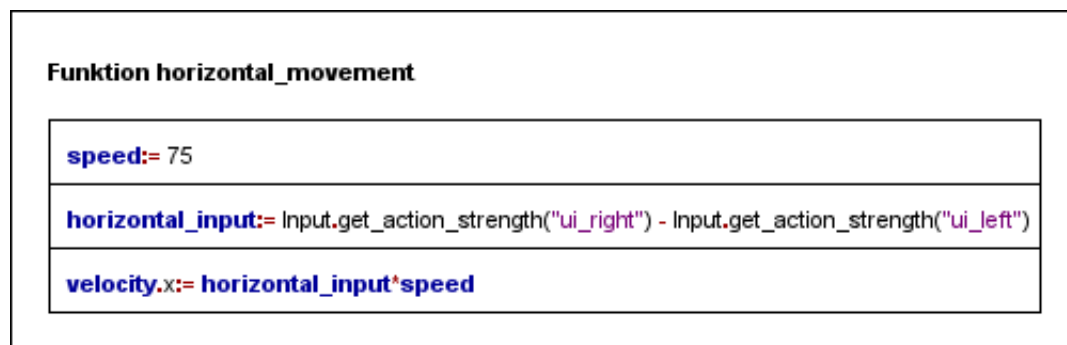


Figure 4: Strukturgramm Horizontales Movement

3.2 Entwurf: Input des Spielers

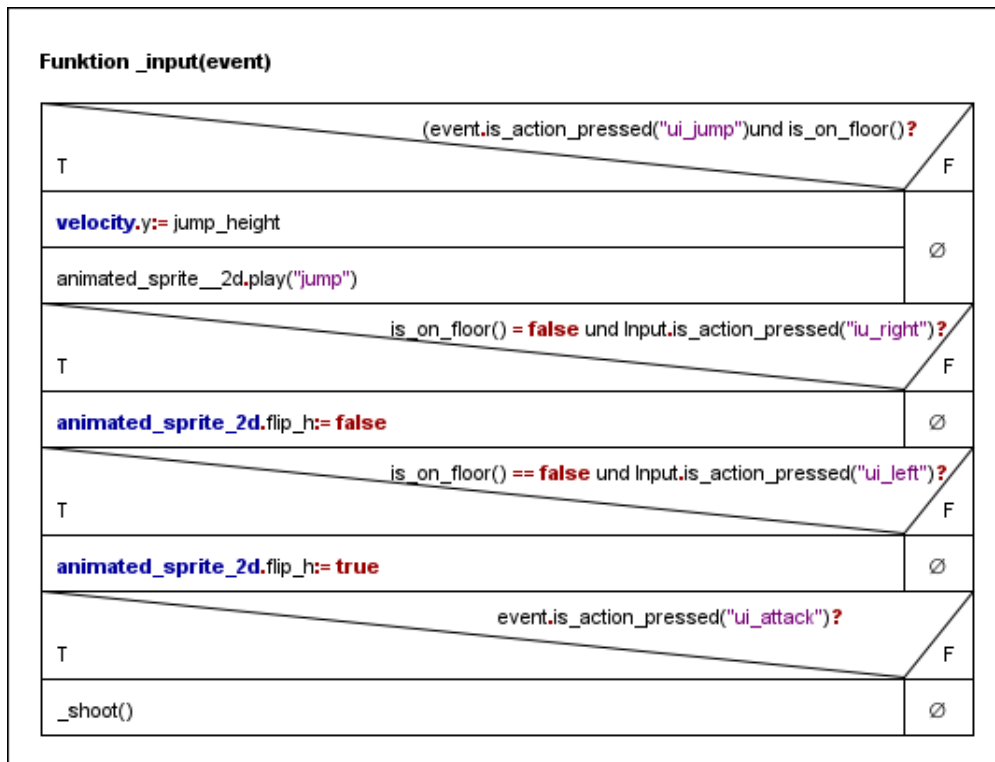


Figure 5: Strukturgramm User Inputs

3.3 Entwurf: Gegnerinteraktion

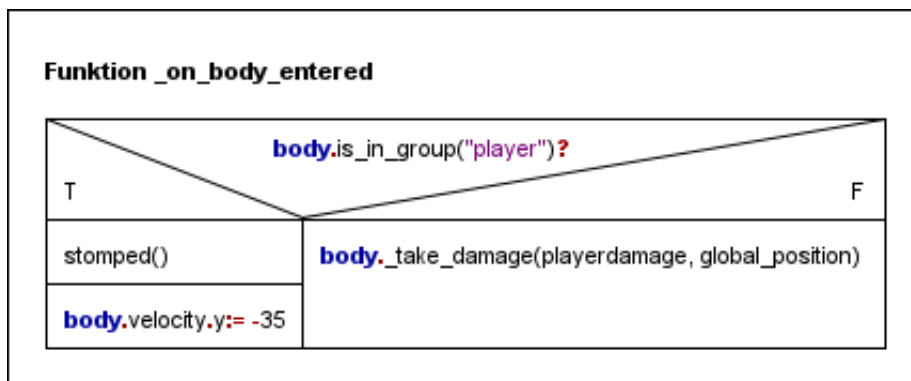


Figure 6: Strukturgramm Gegnerinteraktion

4 Durchführung

In dieser Sektion der Dokumentation wird die aktive Durchführung des Projekts behandelt.

4.1 Beschreiben des Ablaufs der Implementierung

Zu Beginn stand die Ideenfindung im Zentrum. Zum finden einer Idee haben Retro-Spiele als Beispiele im Mittelpunkt gestanden. Zuerst war die Spielidee am originalen Metroid orientiert. Nach weiterem suchen kam die klassischen Mega Man Spiele auf, welche dann zur Hauptidee für das Projekt wurden. Damit waren dann Leveldesingidee und ungefähre HuD Elemente bestimmt.

Für die Implementierung wurde sich für Beginn sehr stark an einem Online-Tutorial orientiert und teilweise Claude Sonnet 4.6. In diesem wird das Erstellen eines simplen Spielers mit Animationen, sowie das Erstellen eines Tylemap-Layers. Als nächstes wurde mit dem selben Tutorial und anderen Online Tutorials eine Kamera hinzugefügt, welche dem Spieler durch das Level folgt. Als nächstes wurde ein Gegner in der Art eines klassischen Koopas aus Super Mario implementiert. Dieser dreht immer an einer Wand oder Abgrund um und läuft in die entgegengesetzte Richtung. Um dem Gegner noch Funktionalität zu bieten, wie das abziehen von Leben, wurde noch eine HuD mit Gamestate Skript für Funktionalität von Leben und Coins hinzugefügt. Dafür wurde hauptsächlich der schon geschriebene Code genutzt, sowie die offizielle Dokumentation von Godot. Auch wurde mit dem Gamestate die erste Funktionalität des Sterbens hinzugefügt. Nachdem die meisten Funktionalitäten für Spieler und Gegner fertig sind, beginnt der Level-Bau. Durch den bereits erstellten TyleMap-Layer war dies nicht schwierig zu implementieren. Jedoch musste noch das zuvor bereits erstellte Level-Ziel eingepflegt werden. Das letzte zu implementierende Feature war ein Checkpoint im Level und das verbinden mit den Coins und dem schon bereits existierenden Respawnpoint. Nach dieser Implementierung und dem erweitern um einen Level Auswahl Screen als Main-Szene, sowie einem Danke am Ende des Spiels, war der Teil des implementierens abgeschlossen.

Während des programmierens und danach, gab es verschiedenste Tests und Spieldurchläufe zum finden von Bugs. Häufig gab es Problematiken mit kleiner Code abschnitten, aber auf Bugs und Fehlerbewältigung wird noch in einem späteren Abschnitt eingegangen.

4.2 Beschreibung des Spielablaufs, Screenshots und Codeausschnitte

Zu Beginn ist der Spieler in einem Level Auswahl Bildschirm. In diesem kann der Spieler dann durch springen in die ausgewählte Box zum gewünschten Level gelangen.

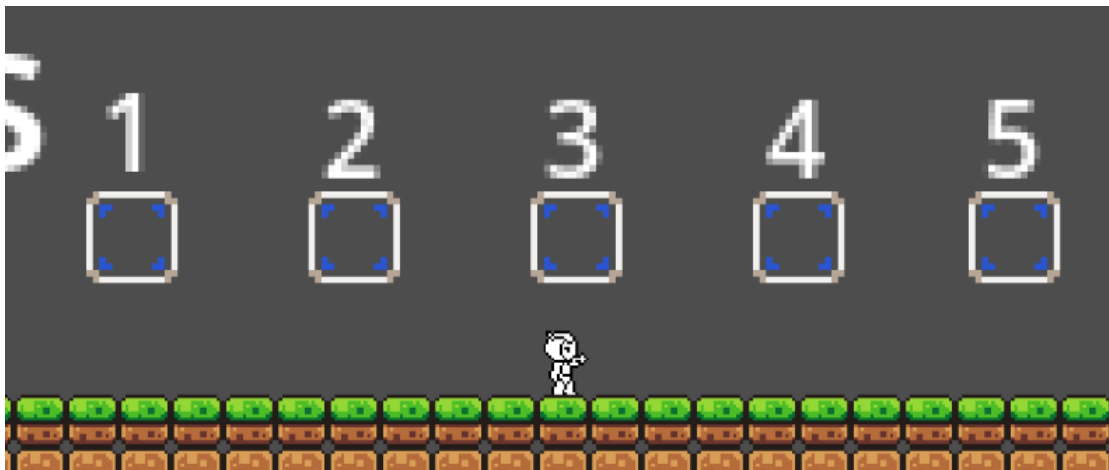


Figure 7: Levelauswahl Bildschirm

Nachdem der Spieler sein Level gewählt hat, wird durch die Box in welcher dieser springt, die Szene des Levels geladen. Da diese Box auch für das Level-Ende genutzt wird, wird diese Interaktion vom levelende.gd Skript gehandhabt. Im Level stehen dem Spieler die grundlegenden Mechaniken des Laufen, Springen und Schießens zur Verfügung. Das Laufen, also das Bewegen auf der x-Achse, ist mit der Funktion `horizontal_movement()` geregelt. In dieser bekommt der Spieler eine Geschwindigkeit zugewiesen und die Richtung wird durch die Inputs des Spielers ermittelt. Aus dem Geschwindigkeitswert und der Richtung wird dann die Geschwindigkeit auf der x-Achse bestimmt.

```

extends Area2D

#Ermöglicht das aendern der Szene im Inspektor
@export var next_level: String = "res://level/level_2.tscn"

func _ready() -> void:

    #Erkennt wenn ein Node2D die Area betritt
    body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node) -> void:
    #Checkt ob Player oder anderer Body Entered
    if body.is_in_group("player"):
        #Ruf die Szene aus next_level auf
        get_tree().call_deferred("change_scene_to_file", next_level)

```

Table 1: Level-ende

```

func horizontal_movement():
    var speed = 75
    # if keys are pressed it will return 1 for ui_right,
    # -1 for ui_left, and 0 for neither
    var horizontal_input = Input.get_action_strength("ui_right")
        - Input.get_action_strength("ui_left")

    #Horizontale Geschwindigkeit bewegt Spieler nach links und rechts
    #Basiert auf User Input
    velocity.x = horizontal_input * speed

```

Table 2: Funktion Horizontales Movement

Um ein vollständiges Jump-and-Run Erlebnis zu bieten, wird noch eine Sprung Funktion benötigt, diese wird in der `_input` Funktion geregelt. In dieser wird die im Player Skript festgelegte Sprunghöhe auf die Geschwindigkeit in y-Richtung angewendet.

```

#Checkt auf User Input und ob der Spieler auf dem Boden ist
if event.is_action_pressed("ui_jump") and is_on_floor():
    #Variable Sprunghoehe setzt die Geschwindigkeit in y-Richtung fest
    velocity.y = jump_height
    #Die Sprites des Spieler startet die Sprung animation
    animated_sprite_2d.play("jump")

```

Table 3: Funktionsabschnitt Sprung

Level 1 ist sehr simpel gehalten und bietet kaum Schwierigkeit um den Spieler in das Spiel rein finden zu lassen. Auch sind direkt am Anfang Münzen platziert, um dem Spieler direkt das Hauptsammelobjekt des Spieles zu präsentieren. Neben den Münzen ist auch eine Plattform in der Luft zu sehen, welche dem Spieler gleich mitteilt, dass es die Möglichkeit gibt Vertikales Movement zu nutzen.

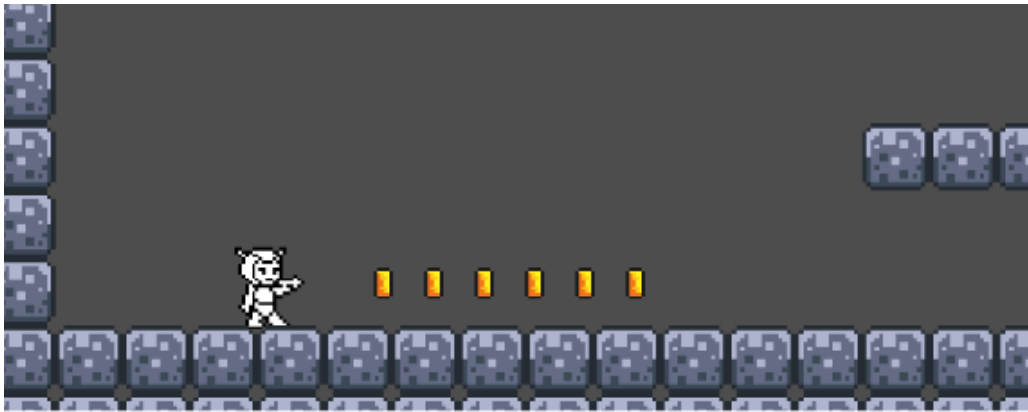


Figure 8: Level 1 Start

Schreitet der Spieler voran, bekommt er einen Gegner zu sehen, sowie einen großen Schriftzug "Press E to Shoot". Damit macht man den Spieler auf die Schuss-Mechanik aufmerksam und zeigt neben der klassischen Jump-and-Run Methode des Heraufspringen noch eine weitere Methode um die Gegner zu besiegen.



Figure 9: Level 1 Erster Gegner

Damit das Schießen überhaupt funktioniert wird eine Schuss Funktion im Player benötigt und ein Skript für das Geschoss selber.

Als Erstes kommt ins Player-Skript die `_shoot()` Funktion. In dieser das Geschoss initialisiert und über einen `Marker2D` dessen Spawn-position bestimmt. Außerdem wird darüber noch ein Offset und die Direction bestimmt, damit diese mit dem Spieler übereinstimmt. Als Letztes wird die Szene als Childnode hinzugefügt und ein Cooldown aktiviert, damit man nicht durchgängig schießen kann.


```

func _shoot() -> void:
    #Checkt ob der Cooldown noch laeuft wenn ja
    #gibt es nichts zurrueck und bricht damit ab
    if shoot_cooldown > 0:
        return

    #Inizialisiert/Laedt die Bullet Szene vor
    var bullet = bullet_scene.instantiate()

    #Setzt die Bullet grundsatzlich an die Marker2D Position
    var offset = muzzle.position
    #Nimmt sich den Absoluten offset also immer positiv
    #Multipliziert den offset mit -1 oder 1 fuer die richtige Richtung
    offset.x = abs(offset.x) * (-1 if animated_sprite_2d.flip_h else 1)
    #Kombiniert die Posiotion der Bullet mit dem offset
    bullet.global_position = global_position + offset
    #Passt die ganze Bullet auf die Richtung an
    bullet.direction = -1 if animated_sprite_2d.flip_h else 1.0
    #Hohlt die Bullet in den Scene Tree als Child um sie abzuspielen
    get_tree().current_scene.add_child(bullet)

    #Setzt den Cooldown neu
    shoot_cooldown = 0.3

```

Table 4: Funktion Schuss des Players

Das Geschoss selber benötigt auch noch ein Skript. Am wichtigsten sind die `_on_body_entered()` und die `_on_area_entered()` Funktionen, welche bestimmen ob das Geschoss mit einem anderen Körper oder einer anderen Area kollidiert. Besonders ist noch die Abfrage nach der Gruppe wichtig, damit die Funktion nur auslöst, wenn ein Gegner getroffen wird.

Das ganze sieht dann Ingame so aus:



Figure 10: Level 1 Gegner Abschießen

Und wenn das Geschoss dann Getroffen hat und der Gegner verschwindet.

```

func _on_body_entered(body: Node) -> void:
    #Beruehrt der Player soll nichts passieren
    if body.is_in_group("player"):
        return
    #beruehrt der Gegner wird die Funktion ausgefuehrt
    if body.is_in_group("enemies"):
        #Funktion des Gegners um ihn zu entfernen
        body.stomped()
        #Animation fuer das Geschoss und das verschwinden davon
        _explode()

func _on_area_entered(area: Area2D) -> void:
    #Hohlt sich die Uebergeordnete Node der Area
    var parent = area.get_parent()
    #Ueberprueft ob Gegner
    if parent.is_in_group("enemies"):
        #Funktion um gegner zu entfernen wird gecallt
        parent.stomped()
        #Animation fuer das Geschoss und das verschwinden davon
        _explode()

```

Table 5: Funktion Schuss des Geschosses



Figure 11: Level 1 Gegner Getroffen und Entfernt

Schreitet man im Level voran und schließt die erste Passage ab, trifft man auf einen Liveball, welcher die Leben des Spielers regeneriert. Falls der Spieler bis dahin bereits Leben verloren hat, wird der Liveball beim einsammeln aktiviert und regeneriert 15 Leben. Hat der Spieler mehr als 85 Leben, so wird auf maximal 100 Leben regeneriert.

Um dies zu realisieren wird ein Gamestate genutzt, welcher die Variablen handhabt, welche von verschiedensten Skripten gebraucht wird. Daneben gibt es noch das Liveball Skript in dem nur die Animationen und die Kollision geregelt werden, deswegen wird auch auf ein Codebeispiel davon verzichtet.



Figure 12: Level 1 Liveball

Wenn das Level abgeschlossen ist, kann man mit den bereits erläuterten Levelenden zum nächsten Level gehen.

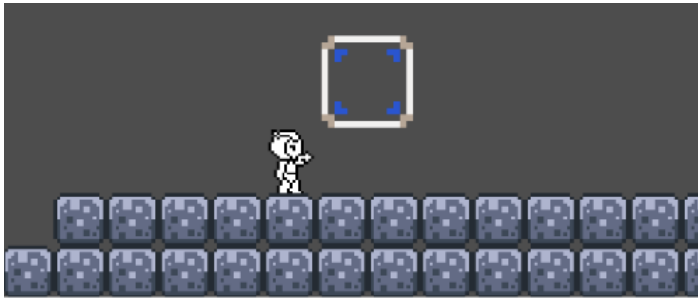


Figure 13: Level 1 Levelende

Im zweiten Level gibt es erst zur Mitte ein neues erwähnenswertes Element. In der Mitte kommt ein Checkpoint zum Einsatz, damit der Spieler bei einem Tod nicht das ganze Level neu spielen muss. Besonderheit am Checkpoint ist, wenn die Münzen auf Null fallen, wird dieser nicht mehr genutzt und der Spieler respawnnt am Levelanfang.

Um dies zu implementieren gibt es ein Checkpoint Skript in dem wieder nur die Kollision und Animation geregelt wird. Das wichtigste läuft im GameState und Player. Der GameState regelt das Verhalten bei Null Münzen und das Resetten der Position des Respawnpoints. Außerdem ist der GameState dafür zuständig die Position des Checkpoints an den Player weiterzugeben. Der Player hat die Methode `_respawn`. In dieser wird in Kombination mit dem GameState die Position des Respawns, den Todesstatus und die Leben beim Respawn geregelt.

```
func _respawn () -> void:
    #Standart Respawn position
    if GameState.respawn_position == Vector2.ZERO:
        #Wird an Ort der Node2D gesetzt
        position = $"../respawnpoint".position
    else:
        #Bekommt ueber den GameState die Checkpoint Koordinaten
        position = GameState.respawn_position

    #Sprite geht immer in Richtung rechts beim Respawn
    animated_sprite_2d.flip_h = false
    #Status fuer den GameState
    GameState.dead = false
    #Setzt Leben wieder auf 100
    GameState.lives = 100
    #Nutzt Signal um die Leben zu uebermitteln
    GameState.lives_changed.emit(GameState.lives)
```

Table 6: Respawn Funktion

Ingame sieht dies dann so aus

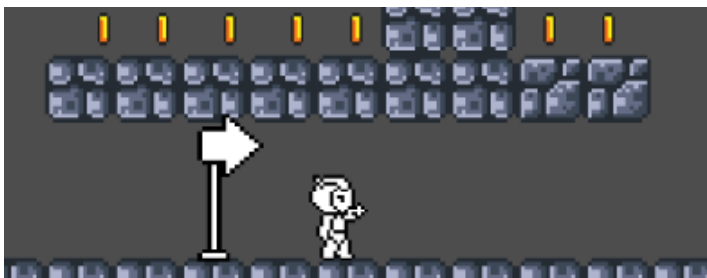


Figure 14: Level 2 Checkpoint

In den Leveln Drei und Vier kommen keine neuen Sachen hinzu. Es sind lediglich weitere Level mit anderen Leveldesign Ideen.

Stil aus Level 3

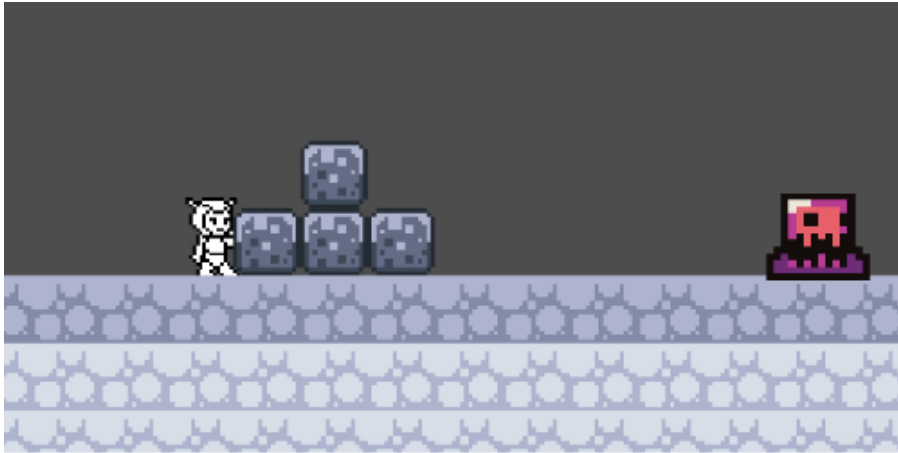


Figure 15: Level 3 Stil

Stil aus Level 4

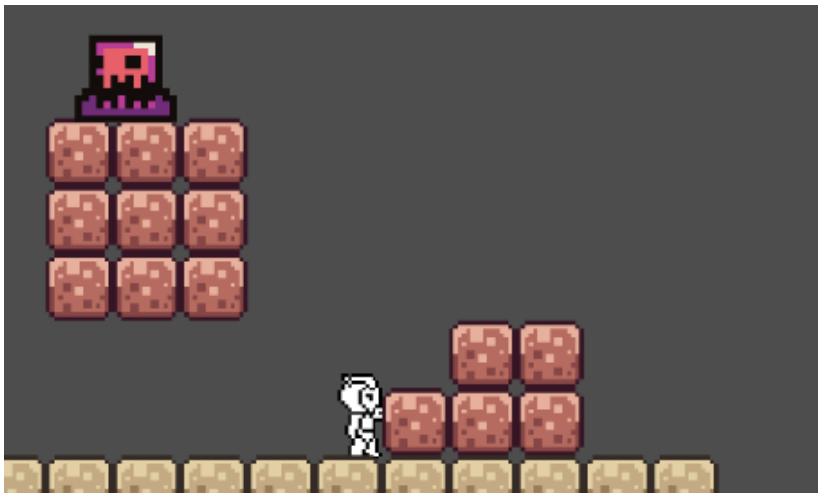


Figure 16: Level 4 Stil

Das letzte Level, also Level 5, bietet eine letzte schwere Herausforderung. Das komplette Level verläuft über dem Abgrund und beinhaltet keine Checkpoints. In diesem Level kommt damit auch die Deatharea besonders auffällig zum Einsatz. Wichtigster Bestandteil davon sind das deatharea Skript, der Gamestate und eine Funktion im Player. Da in diesen Skripts keine neuen Codefunktionen geschrieben werden, werden diese hier nicht weiter erläutert. Die Deatharea erkennt nur das Eintreten eines anderen Bodys aus der Player-Gruppe und ruft die dazugehörige Funktion beim Spieler aus. In dieser wird einfach nur eine Lebensreduktion um Hundert an den Gamestate übermittelt und damit der Spieler dann umgebracht und respawnt.

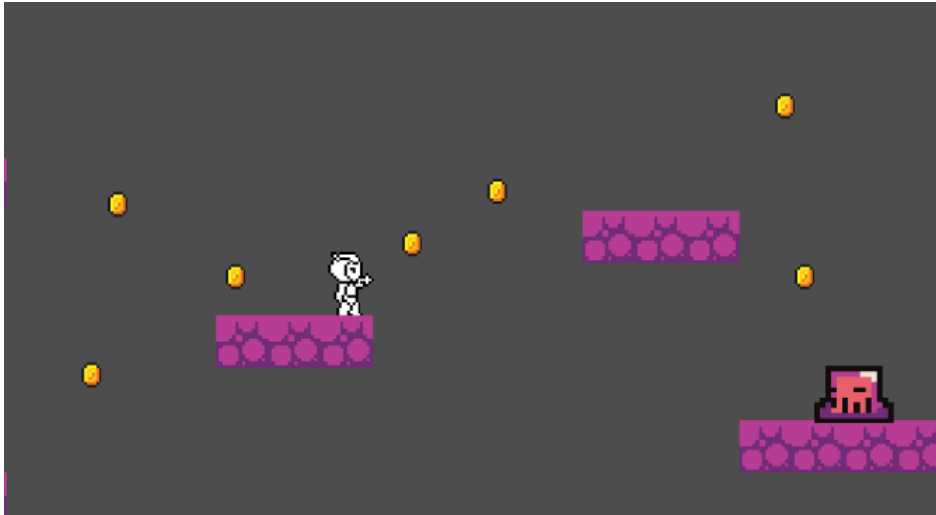


Figure 17: Level 5

4.3 Fehler und Behebung

Einer der auffälligsten Fehler im Spiel war die fehlerhafte Animation, wenn der Spieler gleichzeitig nach Links und nach Rechts Laufen gedrückt hat. Lösen ließ sich das Problem mit einem ändern der Bedingung in den Playeranimationen. Der Code fragte zuerst nur nach Bodenkontakt und Richtung in welche gedrückt wird ab. Dies führte jedoch dazu, dass in diesem Fall dauerhaft eine Laufanimation gespielt wurde, aber der Charakter nicht bewegt wurde. Lösen konnte man das Problem, indem die Bedingung erweitert wurde um eine Abfrage, ob die andere Laufrichtung auch gedrückt ist.

Ein anderes Problem ist bei der Schuss-Mechanik aufgetreten. Drehte sich der Spieler flog das Geschoss weiterhin in die falsche Richtung und verschwand wenn der Spieler getroffen wurde. Das treffen des Spielers konnte einfach gelöst werden, durch eine Abfrage der Gruppe in der die getroffene Node ist. Das Problem mit der Schießrichtung war jedoch komplexer. Mithilfe von Claude Sonnet 4.6 konnte es durch das hinzufügen des Offsets beim Player-Skript gelöst werden, weil dadurch der Marker2D gedreht wurde und auch die Richtung an das Geschoss weitergegeben wurde.

Abgesehen von diesen Fehlern, gab es keine sonderlich erwähnenswerten Fehler.

4.4 Begründung zur Abweichung

Die in der Planung gemachten Pflichtmeilensteine wurden alle erreicht. Damit ist keine Abweichung der ursprünglichen Planung vorhanden. Das Zusatzziel des Schießens auf der y-Achse wurde nicht erreicht. Zum einen durch zeitliche Knappheit und der mit dem Schießen auf der y-Achse verbundenen stark nötigen Änderungen am Leveldesign.

5 Fazit und Ausblick

Das Projekt „Omega Sprinter“ wurde erfolgreich abgeschlossen. Alle geplanten Ziele wurden erreicht: der 2D-Plattformer mit Run-and-Gun-Gameplay, das Strafsystem durch Münzverlust und die Checkpoints funktionieren wie vorgesehen. Das Projekt hat auch gezeigt wie wichtig es ist, realistische Ziele zu setzen und in kleinen Schritten an diesen zu arbeiten.

Falls es in der Zukunft dazu kommen würde an dem Projekt weiter zu arbeiten, wäre eine mögliche Erweiterung, ein richtiger Boss mit mehreren Leben und noch weitere Gegner-arten, welche auf komplexere Fortbewegung setzen.

6 Quellen

Hauptquelle war dieses Online Tutorial:

https://dev.to/christinec_dev/series/23702 Außerdem gab es noch verschieden Prompts an das KI Modell Sonnet 4.6 von Anthropic. Darunter:

- Wie kann ich meinem 2D player in godot 4 eine schuss mechanik geben?
- Wie kann ich meinem 2D player in godot 4 eine schuss mechanik geben?
- Wie kann ich die muzzel flippen die bullet fliegt sonst in den player und verschwindet und einen schuss cooldown
- Wie kann ich den sprite der bullet umdrehen

Als letztes wurde noch die offizielle Godot Dokumentation genutzt.

<https://docs.godotengine.org/de/4.x/>